COURSE GUIDE: PROGRAMMING FOUNDATIONS
PrepWise Synthetic Demo Material - Unit 1: Variables and Expressions
==========

1.1 Memory Model and Assignment Semantics
In Python, an assignment statement does not copy memory bytes. Instead, it binds a variable name to an object reference in memory.
Example:

```
count = 10
total = count
```

Both 'count' and 'total' now point to the integer object 10.

1.2 Dynamic Typing and Primitive Types
Variables do not have fixed static types; values do. The primary types are:
- int: Arbitrary precision signed integers (e.g., -5, 0, 1024).
- float: Double-precision floating point numbers (e.g., 3.14159).
- bool: True or False (subclass of int, where True == 1).
- str: Immutable Unicode character sequences.

1.3 Arithmetic Expressions and Precedence
Operators follow standard precedence: PEMDAS (Parentheses, Exponents **, Multiplication/Division *, /, //, %, Addition/Subtraction +, -).
Integer division (//) truncates toward negative infinity.

==========

2.1 Truthiness and Boolean Evaluation
Every Python value has an implicit boolean value in a conditional context:
- Falsy values: False, None, 0, 0.0, '', [], (), {}.
- Truthy values: Any non-zero number, non-empty string, or collection.

2.2 Comparison Operators and Chaining
Comparisons include == (equality), != (inequality), <, <=, >, >=.
Python supports chained comparisons: 0 <= score <= 100 is equivalent to
(0 <= score) and (score <= 100).

2.3 Multi-Way Branching Structure
The if-elif-else construct evaluates clauses sequentially from top to bottom.
As soon as one branch guard evaluates to True, its block executes and
all subsequent branches are skipped.

2.4 Short-Circuit Evaluation
Logical operators 'and' and 'or' evaluate operands lazily from left to right.
In 'A and B', if A is False, B is never evaluated.
In 'A or B', if A is True, B is never evaluated.

==========

3.1 While-Loop Invariants and Guard Conditions
A while loop repeats its indented body as long as the condition remains True.
The condition is checked at the entry of each iteration, not continuously.
Syntax:

```
while condition:
    # loop body must update state to eventually falsify condition
```

3.2 Sentinel Values and Termination Guarantees
To prevent infinite loops, ensure that loop control variables make monotonic progress toward a termination boundary on every iteration path.
Example of proper boundary check:

```
idx = 0
while idx < len(items):
    process(items[idx])
    idx += 1  # Guaranteed progress
```

3.3 Loop Control: break and continue
- break: Terminates the innermost loop immediately.
- continue: Skips remainder of current iteration and jumps to guard evaluation.

==========

4.1 Anatomical Structure of an Accumulator
An accumulator pattern aggregates data across multiple loop iterations.
It consists of three mandatory elements:
1. Initialization: Set the accumulator variable BEFORE the loop starts.
2. Update: Modify the accumulator inside the loop body using incoming items.
3. Result: Read the accumulated value AFTER the loop completes.

4.2 Summation vs Multiplicative Accumulators
- Sum accumulator must be initialized to the additive identity: 0.
```
total = 0
for x in numbers: total += x
```
- Product accumulator must be initialized to multiplicative identity: 1.
```
product = 1
for x in factors: product *= x
```
(Warning: initializing product to 0 is a fatal zero-product bug!)

4.3 Filtering Accumulators
Combines conditional statements inside the loop to accumulate only matches.
```
evens = []
for num in values:
    if num % 2 == 0:
        evens.append(num)
```

5.1 Variable Scope and Namespaces

Python resolves identifiers using the LEGB hierarchy:
- Local: Names assigned inside the current function.
- Enclosing: Names defined in enclosing function closures.
- Global: Names defined at the module top level.
- Built-in: Predefined identifiers (len, range, sum, int).

5.2 Functions and Return Values

Functions without an explicit 'return' statement implicitly return None.
Be vigilant: printing a value (print(x)) is NOT the same as returning it.

5.3 Assessment Checklist for Foundations Exam

[ ] Verify all while-loop index counters increment on every path.
[ ] Double-check range(start, stop) excludes the stop value.
[ ] Check list indexing bounds: index 0 to len-1 (avoid IndexError).
[ ] Confirm accumulator initialization values (0 for sum, 1 for product).
[ ] Test boundary conditions (empty collections, single-item collections).